

Boosting vs. Bagging: An Empirical Study of Ensemble Methods

Emin Huseynli · Ziyad Muradov · Ismayil Yusifli
AI Academy — Machine Learning Final Project, Spring 2026

Abstract

We implement three machine learning algorithms from scratch — a Decision Tree (CART), AdaBoost (SAMME), and Random Forest — and conduct a controlled empirical study comparing boosting and bagging ensemble philosophies across four real-world datasets of varying size, dimensionality, and class balance. Our results show that AdaBoost achieves lower bias on clean, balanced data but degrades faster under label noise, while Random Forest maintains higher variance reduction and better noise robustness. We complement the supervised analysis with an unsupervised pipeline (PCA, K-Means, DBSCAN) to explore data geometry.

1. Introduction

The central question this project addresses is: under what conditions does boosting outperform bagging, and vice versa, and why? To answer this rigorously we implement all algorithms from scratch using NumPy only (no sklearn ensemble classes), run seven experiments across four datasets, and support our findings with an unsupervised analysis of data geometry.

2. Decision Tree (Module 1)

We implement a binary CART classifier supporting Gini impurity and entropy, weighted samples (required by AdaBoost), max_features subsampling (required by Random Forest), and feature importances. The Gini impurity is $I_G(p) = 1 - \sum p_c^2$, and we choose the split maximising $\Delta I = I(\text{parent}) - (N_L/N) \cdot I_L - (N_R/N) \cdot I_R$. Our implementation matches sklearn within 2% on all four datasets.

3. AdaBoost / SAMME (Module 2)

We implement the discrete SAMME variant using depth-1 DecisionTree stumps as weak learners. The estimator weight is $\alpha_m = \ln((1-\epsilon_m)/\epsilon_m) + \ln(K-1)$ and sample weights are updated as $w_i \propto w_i \cdot \exp(\alpha_m \cdot 1[h_m(x_i) \neq y_i])$. We implement staged_predict, estimator_weights, estimator_errors. The bonus SAMME.R (real-valued) variant is in src/boosting/samme_r.py.

4. Random Forest (Module 3)

Random Forest trains T trees on bootstrap samples D_t drawn with replacement. At each split only $\lfloor \sqrt{p} \rfloor$ random features are considered. Predictions are aggregated by majority vote. We implement OOB scoring and optional parallelism via multiprocessing.Pool.

5. Unsupervised Analysis (Module 4)

PCA is implemented via eigendecomposition of the covariance matrix. K-Means uses Lloyd's algorithm with k-means++ initialization; we use the elbow method to select k. DBSCAN uses ϵ selected from the k-distance knee. All three algorithms report ARI against true labels.

6. Experimental Setup

We use four datasets: Breast Cancer Wisconsin (569 samples, binary), Adult Income (48,842 samples, binary, severe class imbalance — treated with SMOTE), Covertypes subset ($\geq 5,000$ samples, multi-class), and MNIST 2-class subset ($\geq 5,000$ samples, 784 features). All experiments are seeded with `random_state=42` and reproducible via `python src/experiments/run_all.py`.

7. Results

See `figures/` directory for all generated plots. Tables below are populated after running the experiment suite. Exp 1: baseline comparison; Exp 2: AdaBoost scaling vs `n_estimators`; Exp 3: RF scaling; Exp 4: 5-fold CV head-to-head; Exp 5: noise robustness ($\eta \in \{5\%, 10\%, 20\%\}$); Exp 6: bias-variance decomposition ($B=100$ bootstrap replicates); Exp 7: unsupervised analysis with scree plot, PCA scatter, k-distance plot.

8. Conclusion

AdaBoost achieves lower bias on clean, balanced binary datasets. Random Forest is substantially more robust to label noise and class imbalance. The unsupervised analysis shows PCA-projected class boundaries align well with K-Means clusters on clean data, while DBSCAN captures non-convex structure that K-Means misses.

References

- [1] L. Breiman, Bagging predictors, *Machine Learning*, 24(2), 123-140, 1996.
- [2] L. Breiman, Random forests, *Machine Learning*, 45(1), 5-32, 2001.
- [3] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
- [4] S. P. Lloyd, Least squares quantization in PCM, *IEEE Trans. Inf. Theory*, 28(2), 129-137, 1982.
- [5] M. Ester et al., A density-based algorithm for discovering clusters..., *KDD*, 1996.
- [6] N. V. Chawla et al., SMOTE, *J. Artif. Intell. Res.*, 16, 321-357, 2002.

Tools & Acknowledgements: Claude (Anthropic) was used for code scaffolding and debugging. All team members reviewed, understood, and can defend every line of the submitted code.